

NEXTRACE

CI/CD 공급망 침해 영향도 분석 및 AWS 자산 추적 플랫폼

2팀 추적자 - 정○○(C) · 백○○(P) · 김○○ · 이준서

Mentor - 홍○○ (S○○○○) · 유○○ (원○○○)

목차

Online Security Basics

PART 01

팀원 소개 및 일정

PART 02

주제 선정 배경

PART 03

프로젝트 소개

PART 04

핵심 기능

PART 05

분석 로직

PART 06

시연 영상

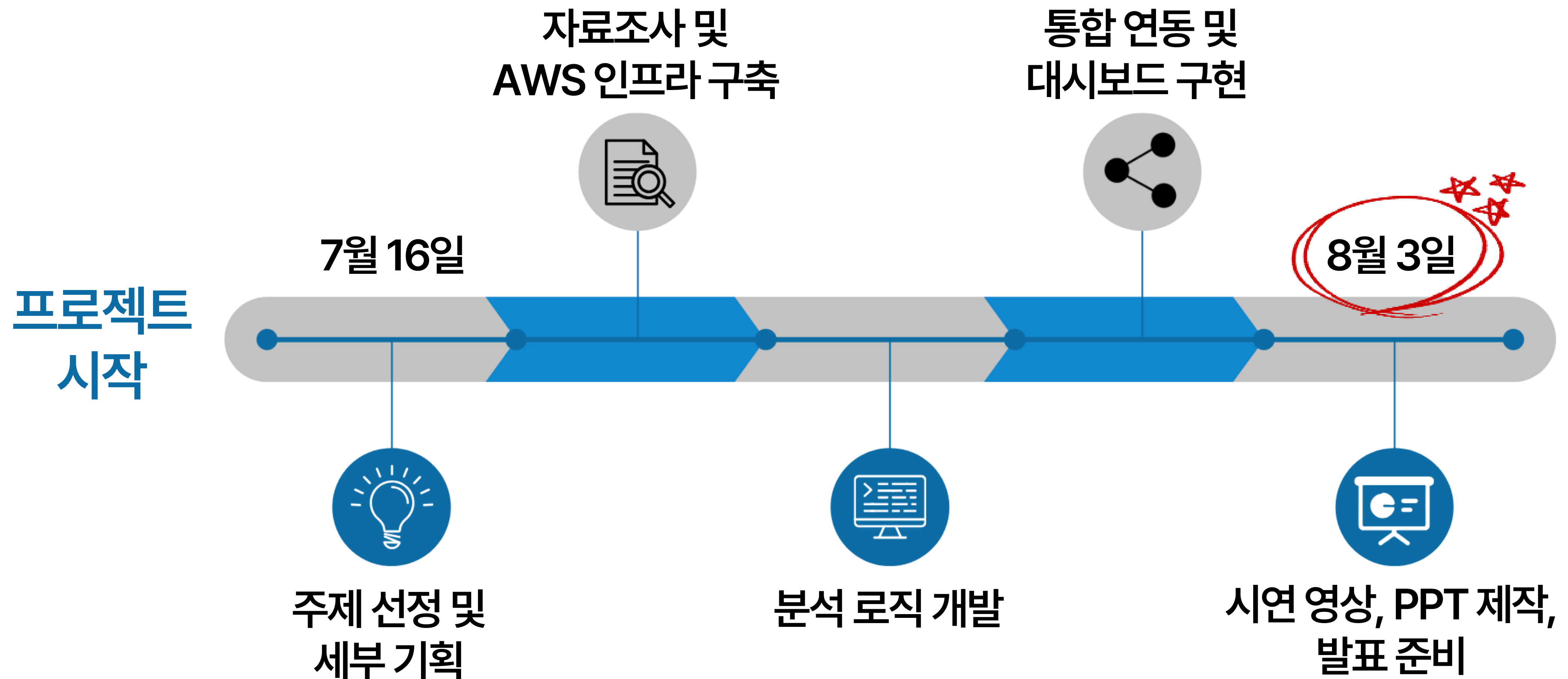
PART 07

개선 사항 및 마무리

팀원 소개 및 일정

일정

Project Timeline





SECURITY NEXTRACE

SYSTEM STATUS

SIEM Active	XDR Active	EDR Active
Firewall Active	Cloud Active	DB Active

ALERT FEED

- [CRITICAL]** Suspicious Login Detected
10.15.20.5 | User: admin | 14:22:01
- [HIGH]** Malware Detected
server-03 | Trojan.Win32 | 14:21:10
- [HIGH]** Unusual Data Exfiltration
192.168.1.45 | 1.2GB | 14:20:05
- [MEDIUM]** Privilege Escalation Attempt
dev-user | sudo access | 14:19:42
- [LOW]** Port Scan Detected
203.0.113.7 | 22, 80, 443 | 14:18:33



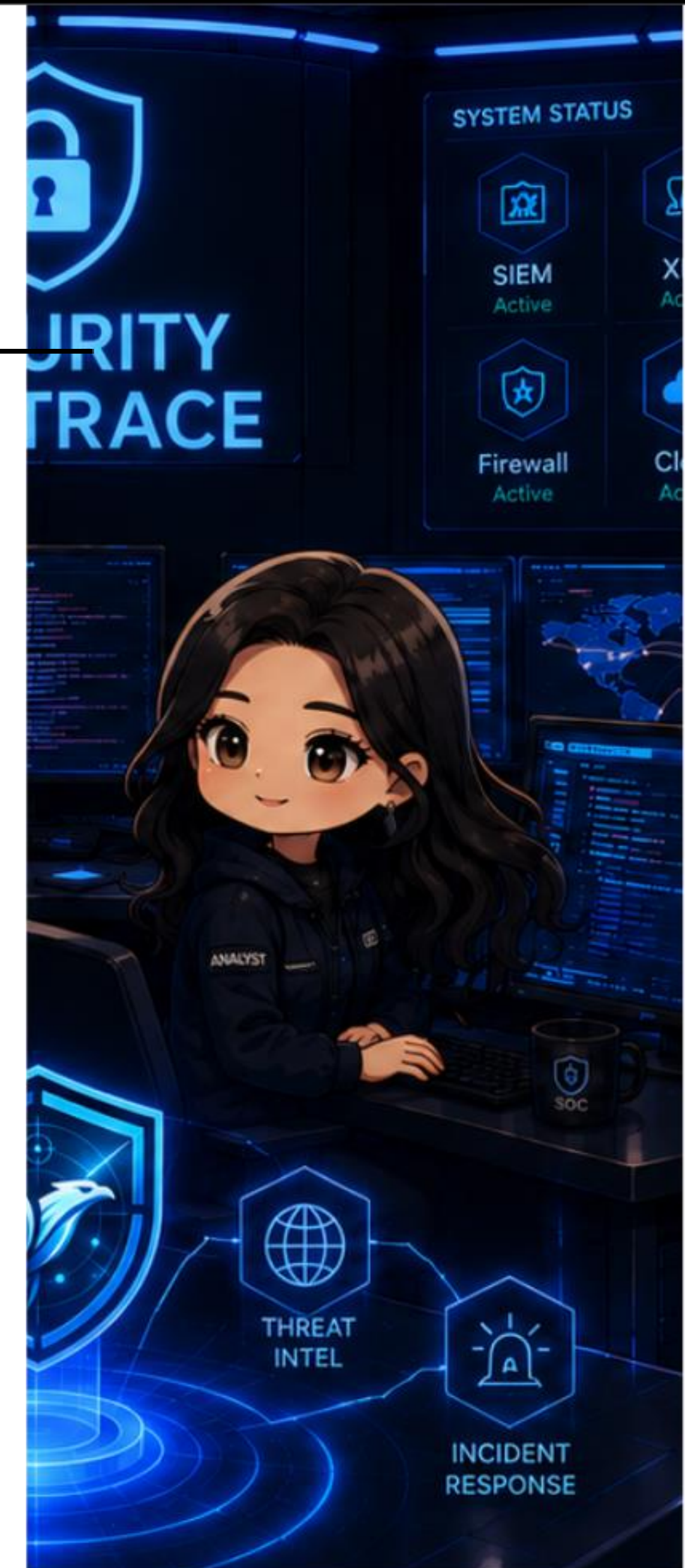
정○○○ (C) Data Engineer

- 프로젝트 관리 및 역할 조율
- 분석 데이터 구조 및 DB 모델 설계
- 분석 결과 저장 및 Dashboard
데이터 구성
- Workflow 연계 데이터 관리
- 시연 영상 제작



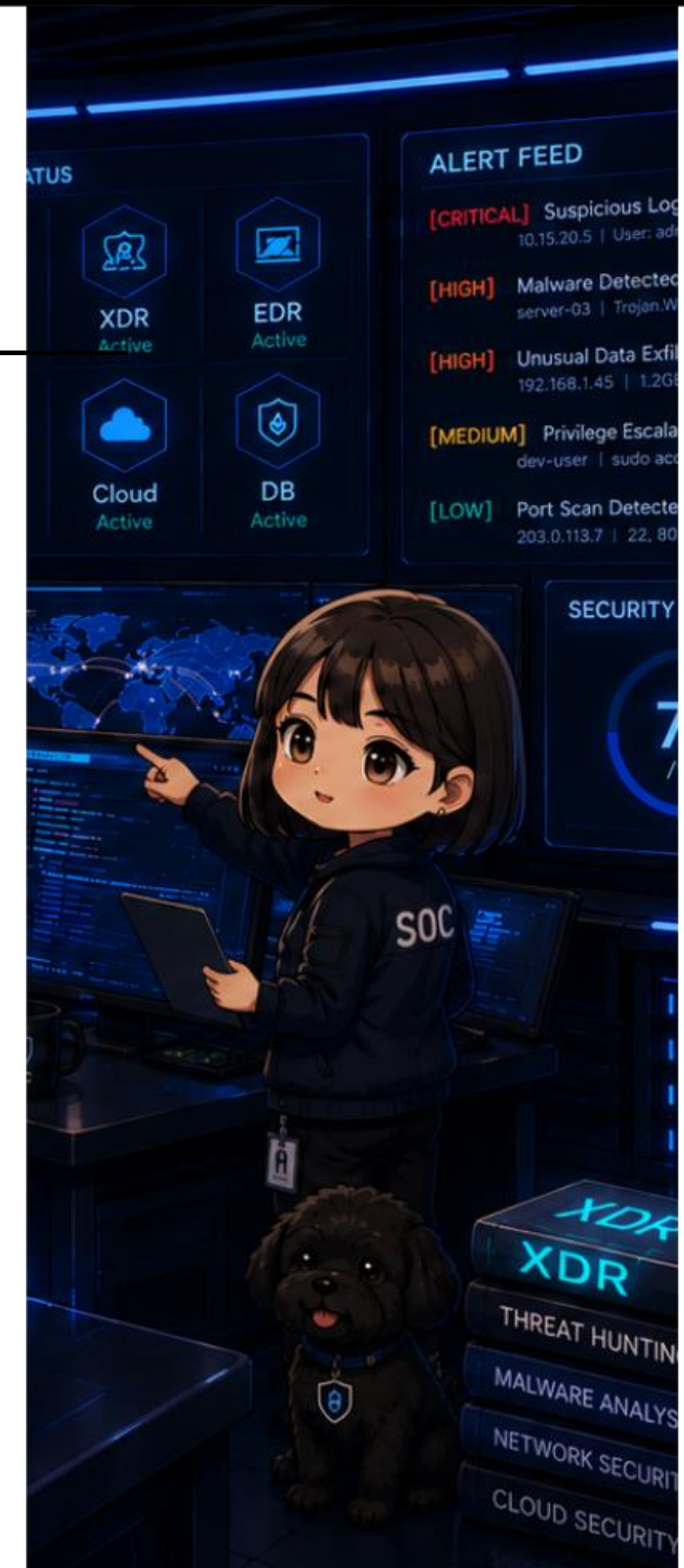
백○○○ (P) Security Analysis Engineer

- 분석 엔진 및 판정 파이프라인 구현
- CloudTrail 상관분석
- 위험 행동 분류 및 점수 계산
- 최종 Incident 생성
- 프로젝트 발표



김○○ Frontend · Dashboard Engineer

- Dashboard UI 설계 및 구현
- Dashboard API 연동 및 데이터 처리
- Incident · Attack Path · Timeline 시각화
- 보안 관제 기능 구현 및 PPT 작성



이준서 Cloud · Devops Engineer

- AWS 아키텍처 설계 및 인프라 구축
- GitHub Actions Workflow 구축
- PPT 보조



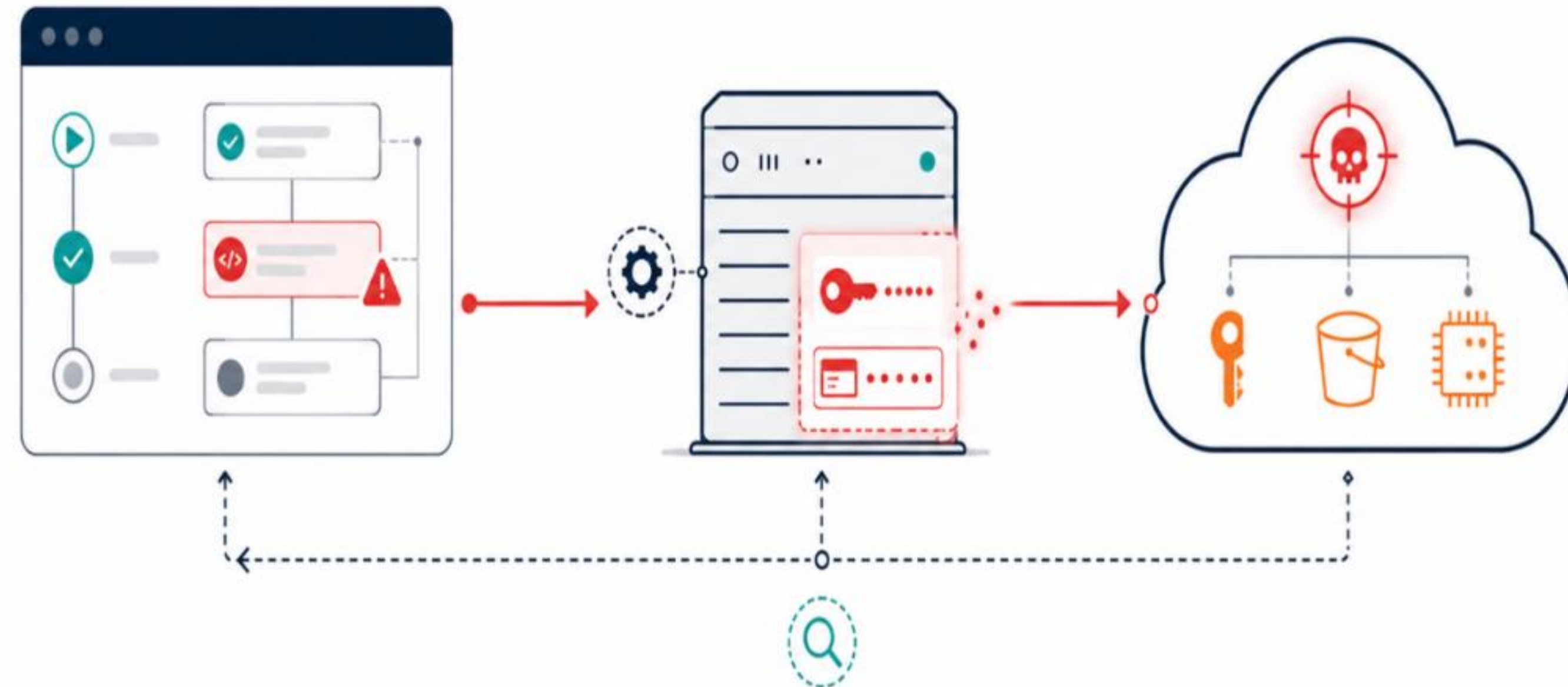
주제 선정 배경

주제 선정 배경 - 신뢰받던 Github Action이 공격 경로가 되다

긴급속보

[긴급] 깃허브 액션스 공급망 공격 발생...218개 저장소 비밀정보 유출돼

공격 진원지는 아직 조사 중...다른 Action도 침해됐을 가능성 있어



전 세계 개발자들이 널리 사용하는 자동화 도구인 깃허브(GitHub) Actions에서 공급망 공격이 발생해, 최소 218개 오픈소스 저장소에서 민감한 비밀정보가 외부에 유출된 것으로 확인됐다. 특히 이들 중 일부는 수십만 개의 스타를 받은 인기 프로젝트로, 파급력이 클 수 있다는 분석이다.

이번 공격은 2025년 3월 14일, 깃허브(GitHub) Action 중 하나인 tj-actions/changed-files에 악성 커밋이 삽입되며 시작됐다. 공격자는 CI/CD(지속적 통합 및 지속적 배포) 파이프라인의 러너(Runner)에서 동작하는 워크플로우를 악용해 민감한 비밀정보(Secrets)를 레포지토리 콘솔 로그에 출력되도록 조작했다. 로그가 공개 설정일 경우 누구나 해당 정보를 열람할 수 있는 상태였다.

주제 선정 배경 - CI/CD에서 시작된 침해는 클라우드 전체로 확산된다

해외

EU 집행위 클라우드 해킹당해, 30개 가까운 유럽연합 기관으로 확산

TeamPCP, 탈취한 AWS API 키로 침투...이메일·개인정보 포함 수만 건 파일 반출
공용 클라우드 운영 구조 노린 연쇄 공격...내부 42개 고객과 외부 29개 기관 영향 가능성



2026년 3월, 변조된 Trivy를 통해 AWS API Key가 공격자에게 탈취

- > STS 호출 및 추가 Secret 탐색
- 기존 IAM 사용자의 신규 Access Key 생성
- AWS 자산 정찰 및 데이터 접근
- 약 91.7GB의 데이터 유출

유럽연합 집행위원회(EC) 클라우드 환경이 해킹당하면서 집행위 내부 조직뿐 아니라 다른 유럽연합 기관들까지 연쇄적으로 영향을 받은 사실이 드러났다. 유럽연합 사이버보안 대응 조직인 CERT-EU는 이번 사고의 배후로 TeamPCP를 지목하고, 이 침해로 최소 29개 다른 유럽연합 기관 데이터까지 노출됐을 가능성이 있다고 밝혔다.

이번 사건은 지난 3월 말 처음 공개됐다. 집행위는 유럽연합 공식 웹서비스를 운영하는 아마존웹서비스(AWS) 기반 클라우드 환경에서 침해 정황을 확인했다고 밝혔고, 이후 후속 분석에서 피해 범위가 예상보다 더 넓은 것으로 나타났다. CERT-EU에 따르면 공격자는 3월 10일, 앞선 트리비(Trivy) 공급망 공격 과정에서 탈취한 AWS API 키를 이용해 집행위 클라우드 환경에 침입했다.

주제 선정 배경 - 이 사건이 보여준 기존 관제의 한계

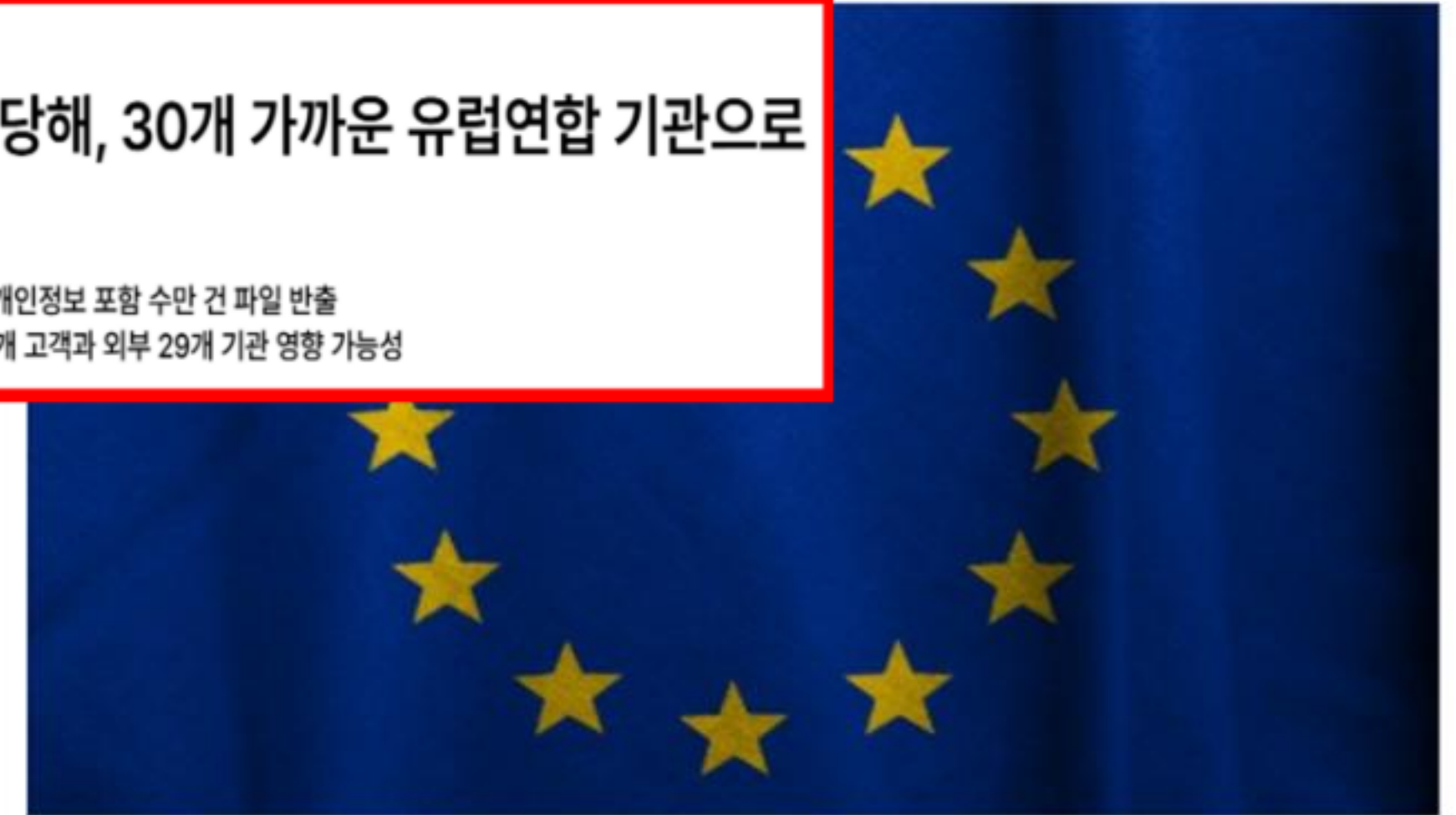
2026년 3월, 변조된 Trivy를 통해 AWS API Key가 공격자에게 탈취

- > STS 호출 및 추가 Secret 탐색
- 기존 IAM 사용자의 신규 Access Key 생성
- AWS 자산 정찰 및 데이터 접근
- 약 91.7GB의 데이터 유출

해외

EU 집행위 클라우드 해킹당해, 30개 가까운 유럽연합 기관으로 확산

TeamPCP, 탈취한 AWS API 키로 침투...이메일·개인정보 포함 수만 건 파일 반출
공용 클라우드 운영 구조 노린 연쇄 공격...내부 42개 고객과 외부 29개 기관 영향 가능성



유럽연합 집행위원회(EC) 클라우드 환경이 해킹당하면서 집행위 내부 조직뿐 아니라 다른 유럽연합 기관들까지 연쇄적으로 영향을 받은 사실이 드러났다. 유럽연합 사이버보안 대응 조직인 CERT-EU는 이번 사고의 배후로 TeamPCP를 지목하고, 이 침해로 최소 29개 다른 유럽연합 기관 데이터까지 노출됐을 가능성이 있다고 밝혔다.

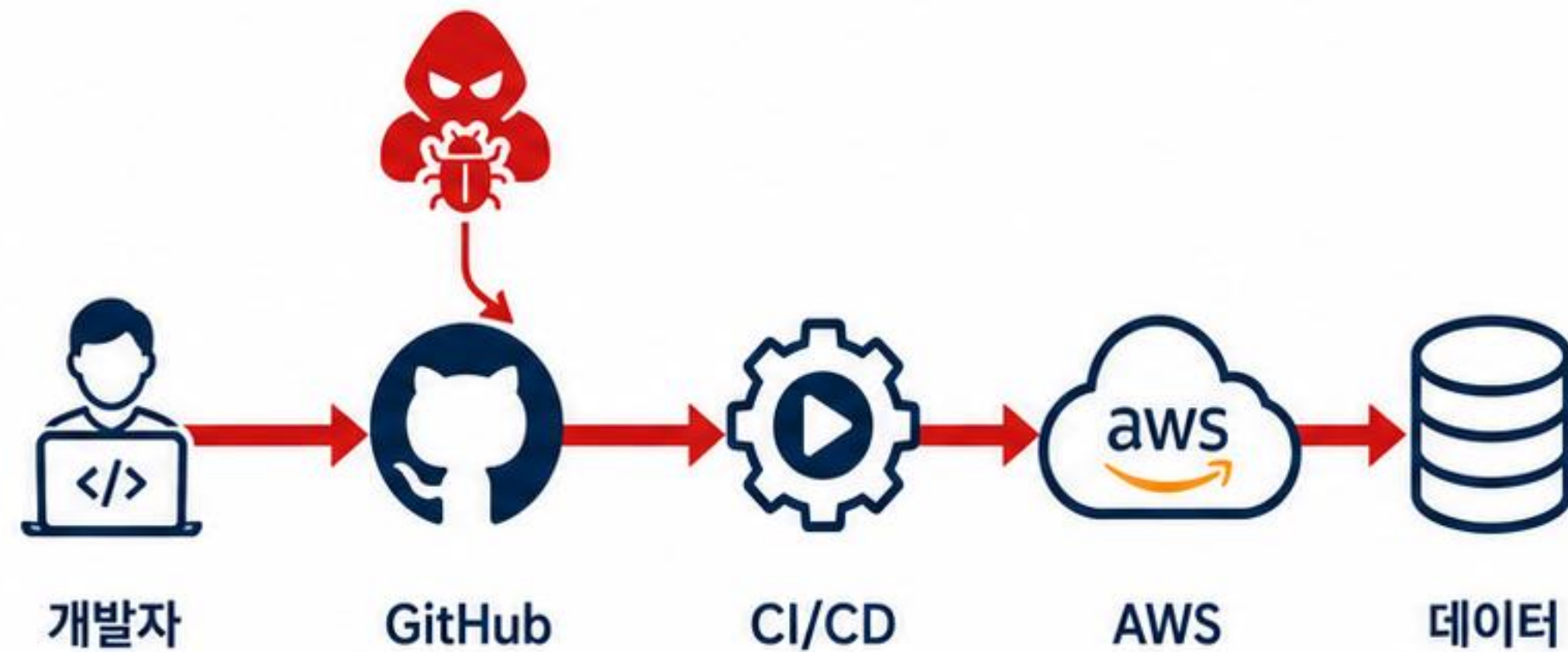
이번 사건은 지난 3월 말 처음 공개됐다. 집행위는 유럽연합 공식 웹서비스를 운영하는 아마존웹서비스(AWS) 기반 클라우드 환경에서 침해 정황을 확인했다고 밝혔고, 이후 후속 분석에서 피해 범위가 예상보다 더 넓은 것으로 나타났다. CERT-EU에 따르면 공격자는 3월 10일, 앞선 트리비(Trivy) 공급망 공격 과정에서 탈취한 AWS API 키를 이용해 집행위 클라우드 환경에 침입했다.

의미 & 시사점

- CI/CD 침해는 클라우드 자산 전체로 확산될 수 있다.
- 분산된 로그만으로는 공격 경로와 피해 범위 파악에 한계가 있다.
- 공격의 시작부터 최종 영향 자산까지 연결 추적할 체계가 필요하다.

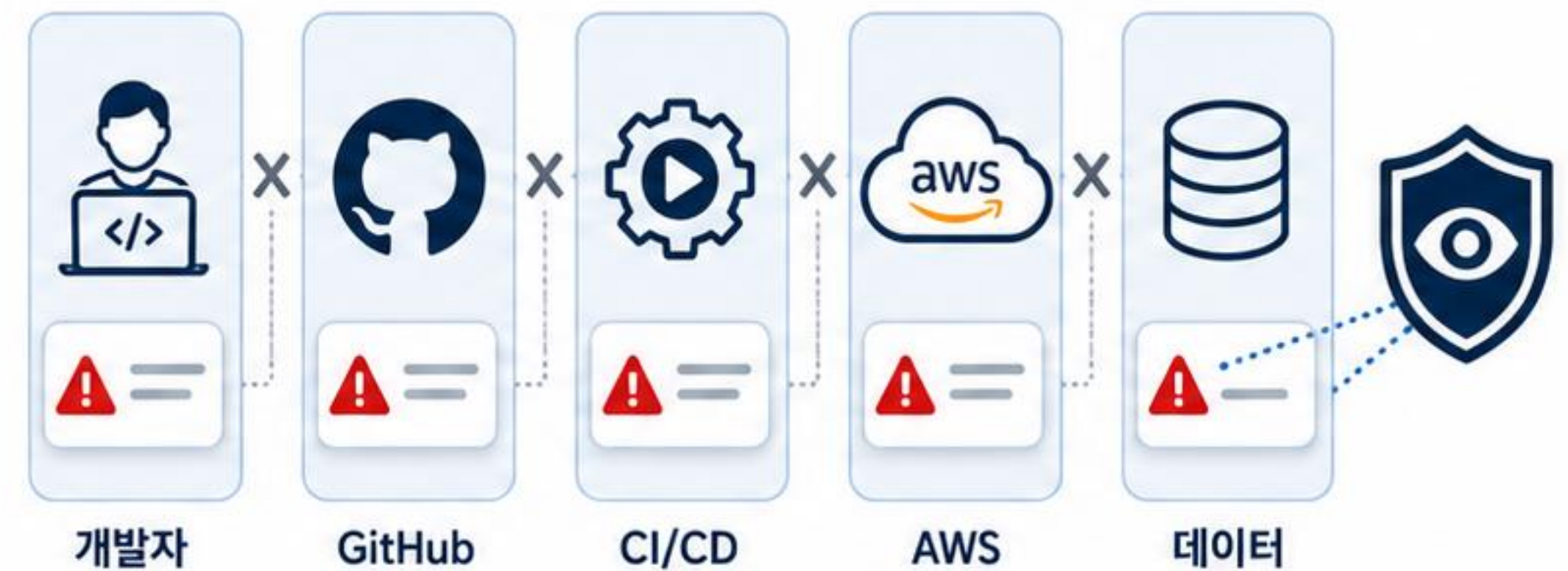
주제 선정 배경 - CI/CD 공급망 침해 위험 · 기존 관제의 한계

연결된 공격



공격은 파이프라인 전체로 확산

분리된 기존 관제



개별 경고만 확인 · 전체 흐름 파악 어려움

프로젝트 소개

"CI/CD에서 클라우드까지, 침해 경로를 추적하다"



CI/CD 공급망 침해로부터 AWS 자산 영향까지 연결하는
"통합 위협 추적 플랫폼"

프로젝트 목표 및 기획 의도

흩어진 흔적을 하나의 공격 흐름으로



공격 흔적 **통합**

CI/CD · CloudTrail 로그 연계



공격 타임라인 **재구성**

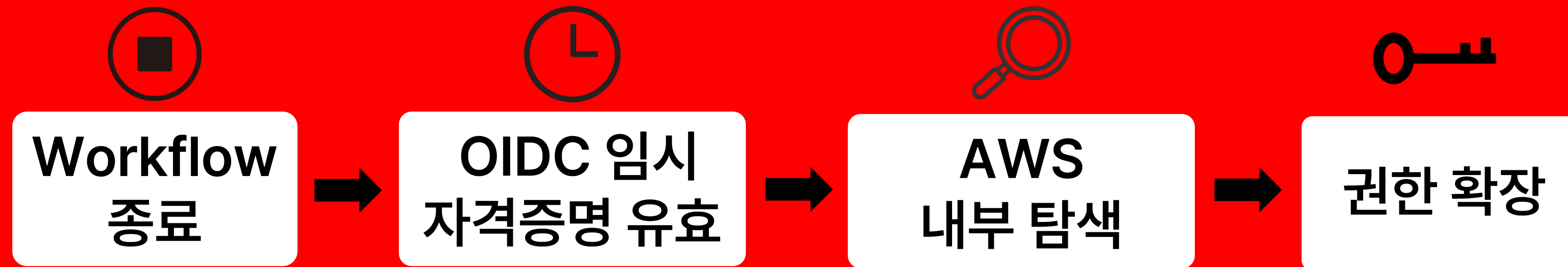
위험 Action 연관성 분석



침해 영향 범위 **시각화**

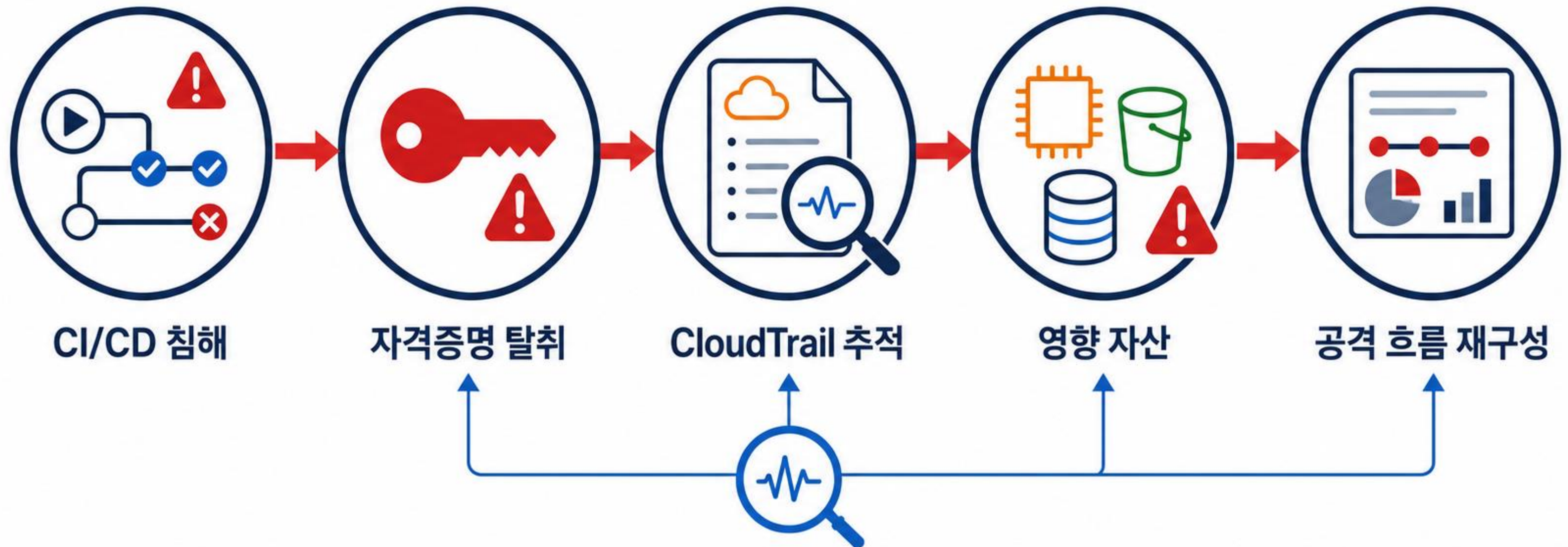
접근 · 권한 자산 추적

프로젝트 문제 정의

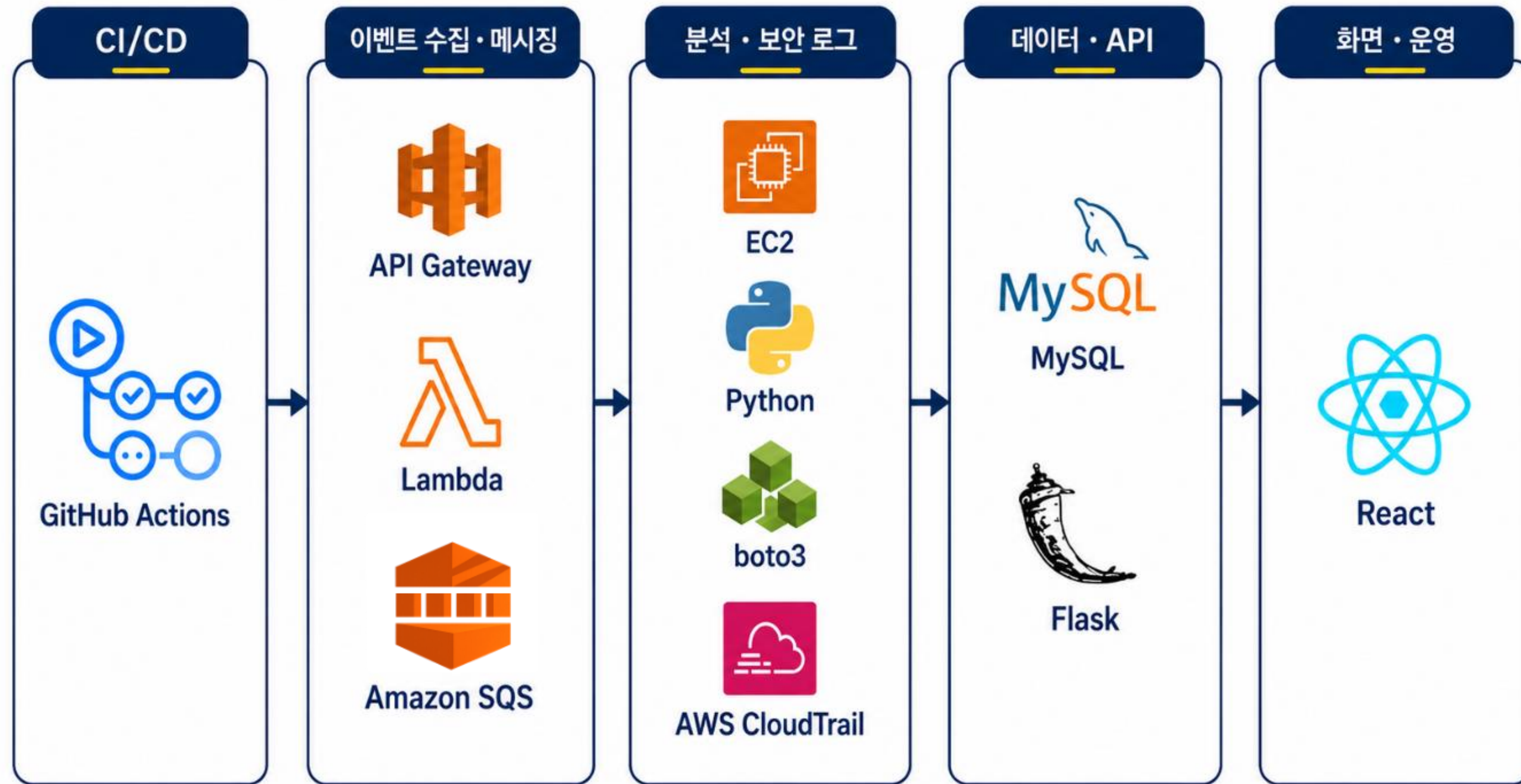


OIDC 임시 자격증명은 만료 전까지 AWS 내부 탐색과 권한 확장에 악용

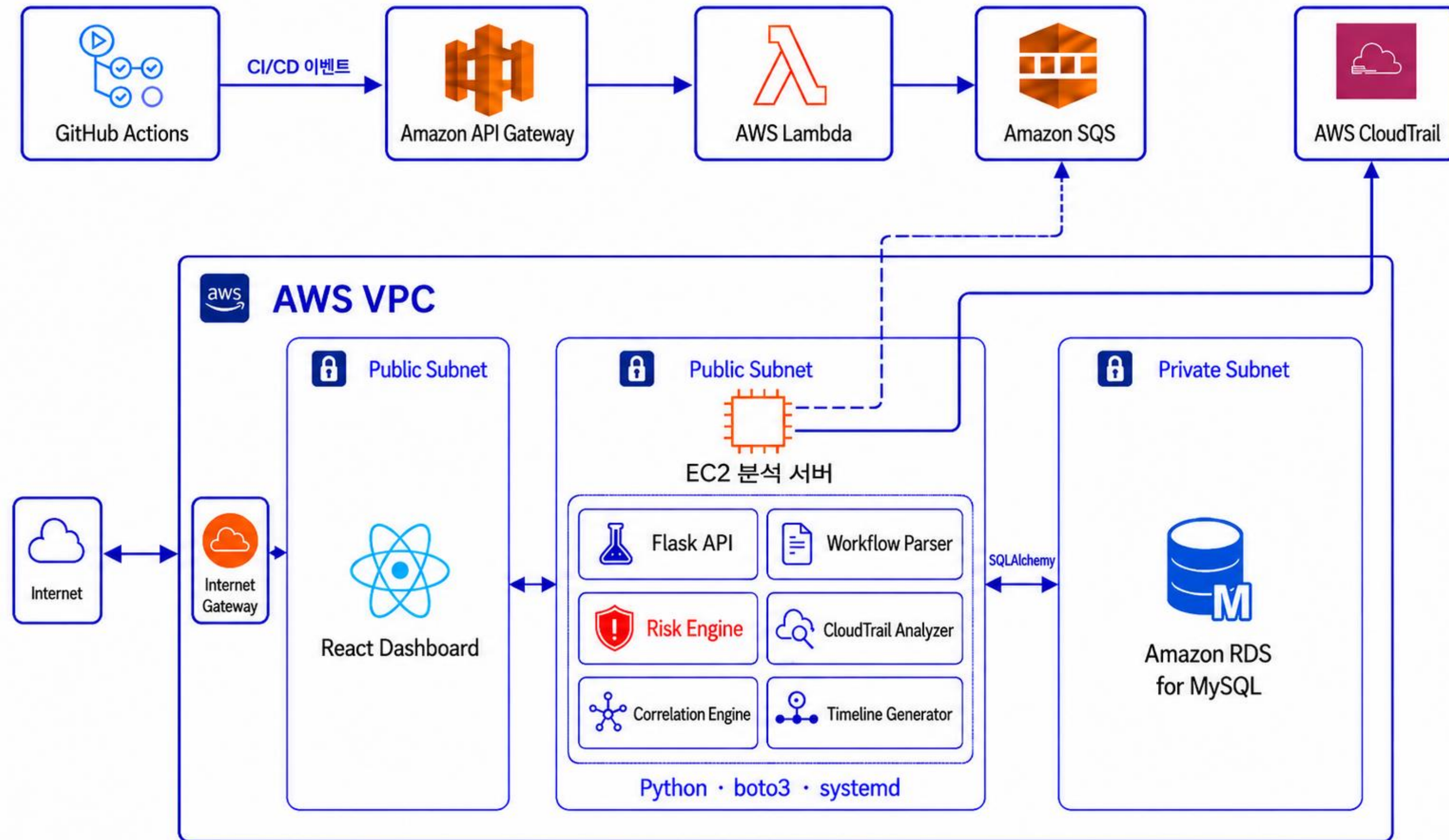
프로젝트 필요성 - 침해의 시작과 영향을 하나로 연결



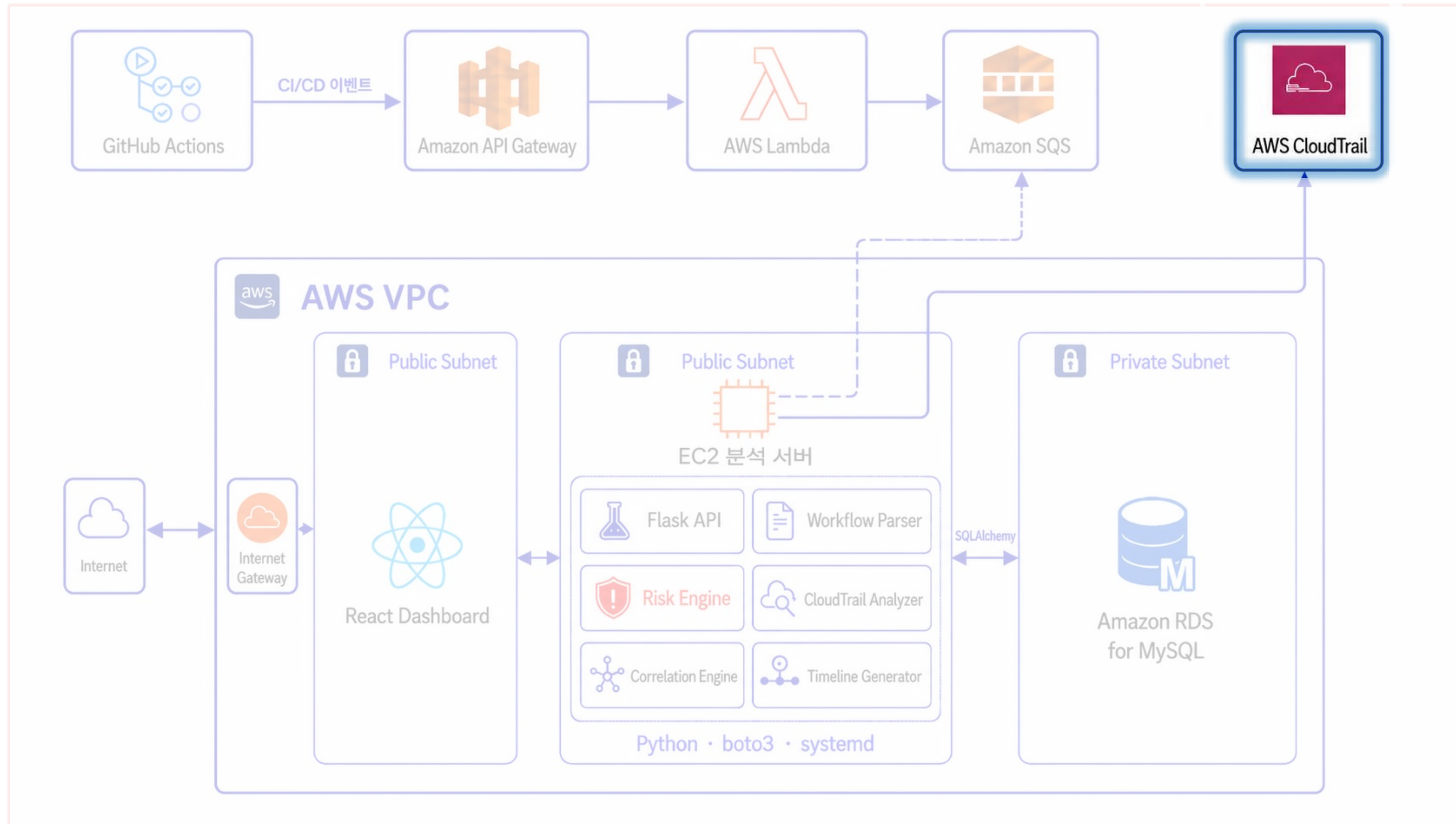
기술 스택



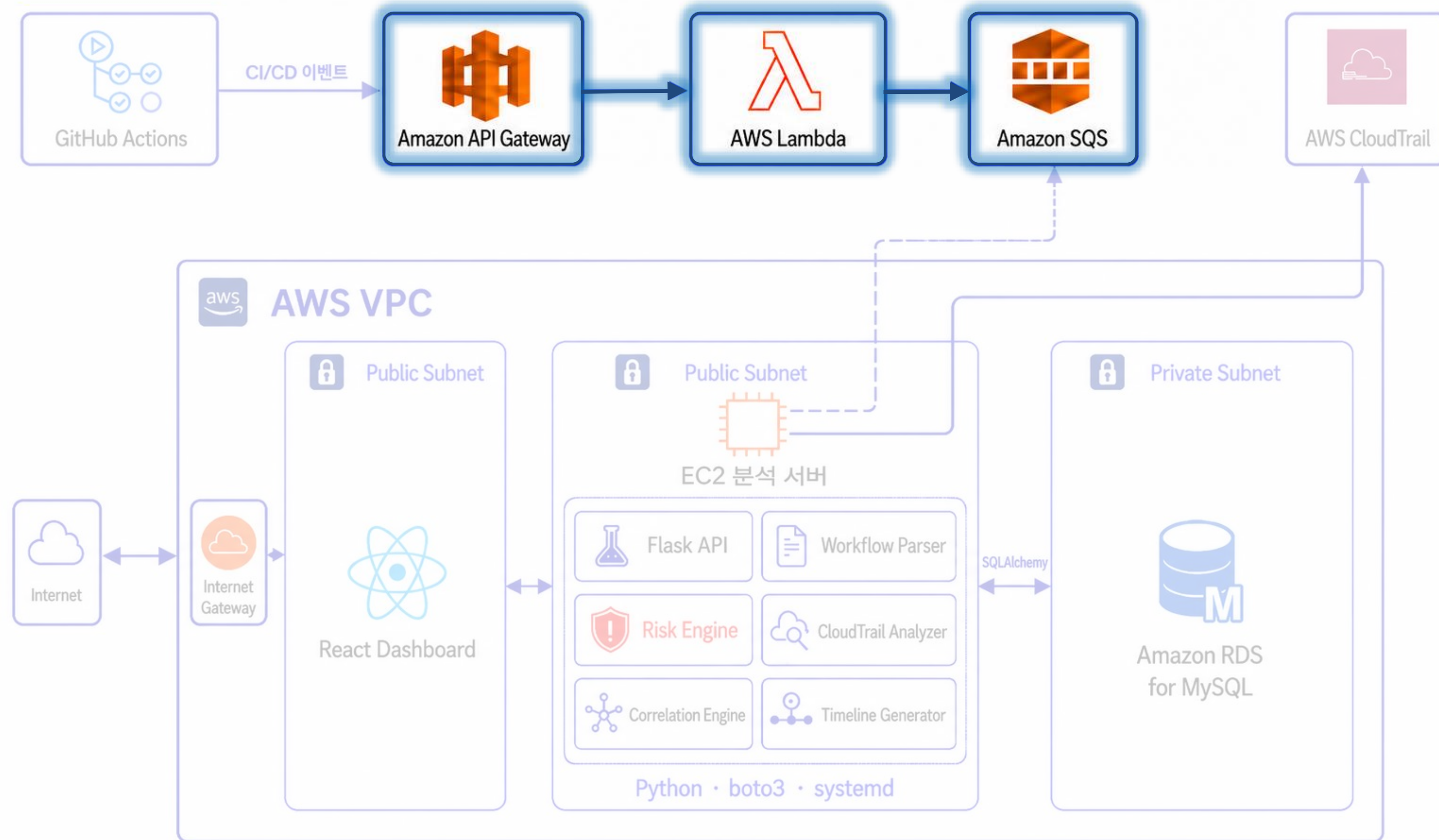
NEXTRACE 전체 아키텍처



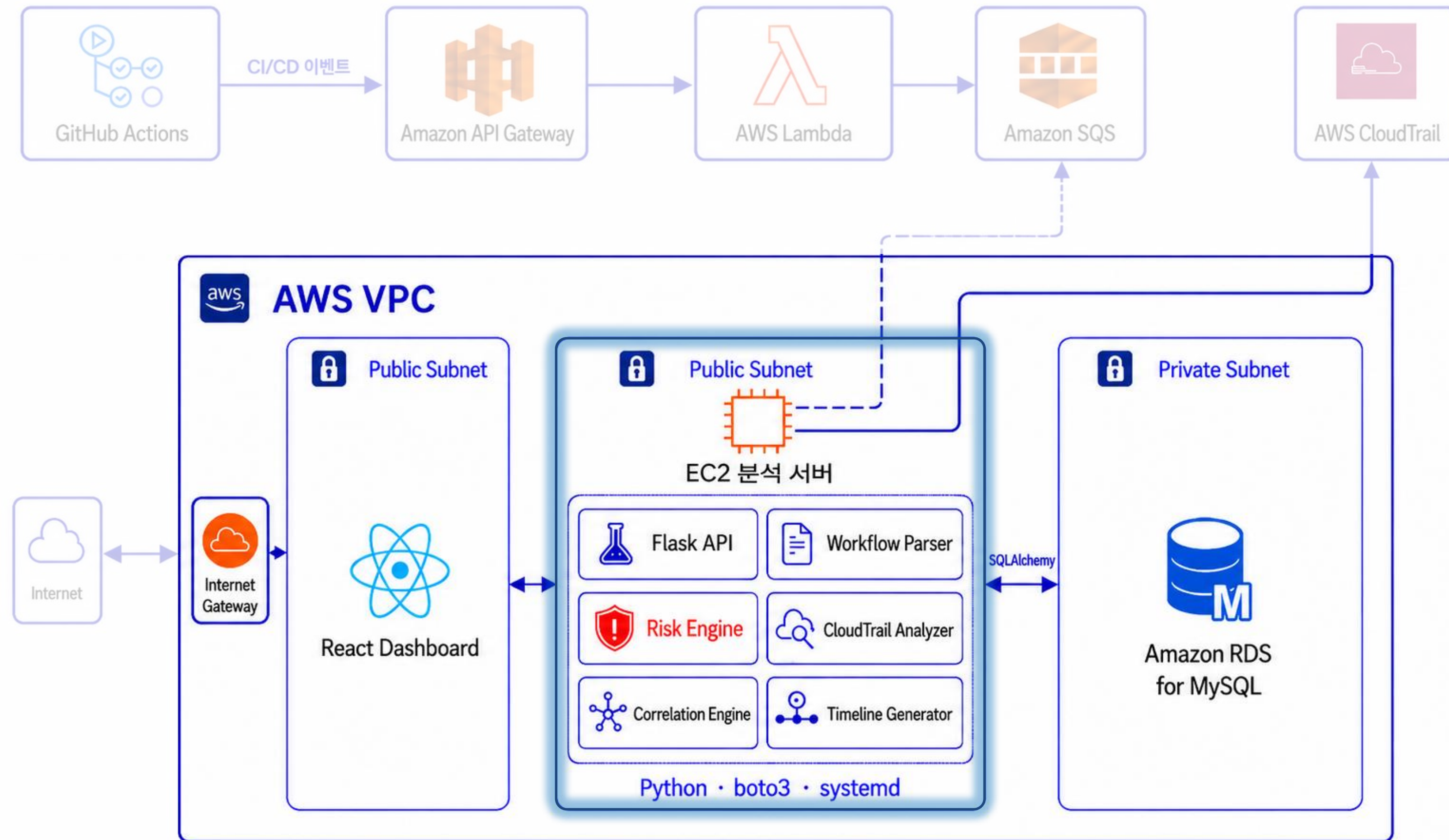
NEXTRACE 전체 아키텍처



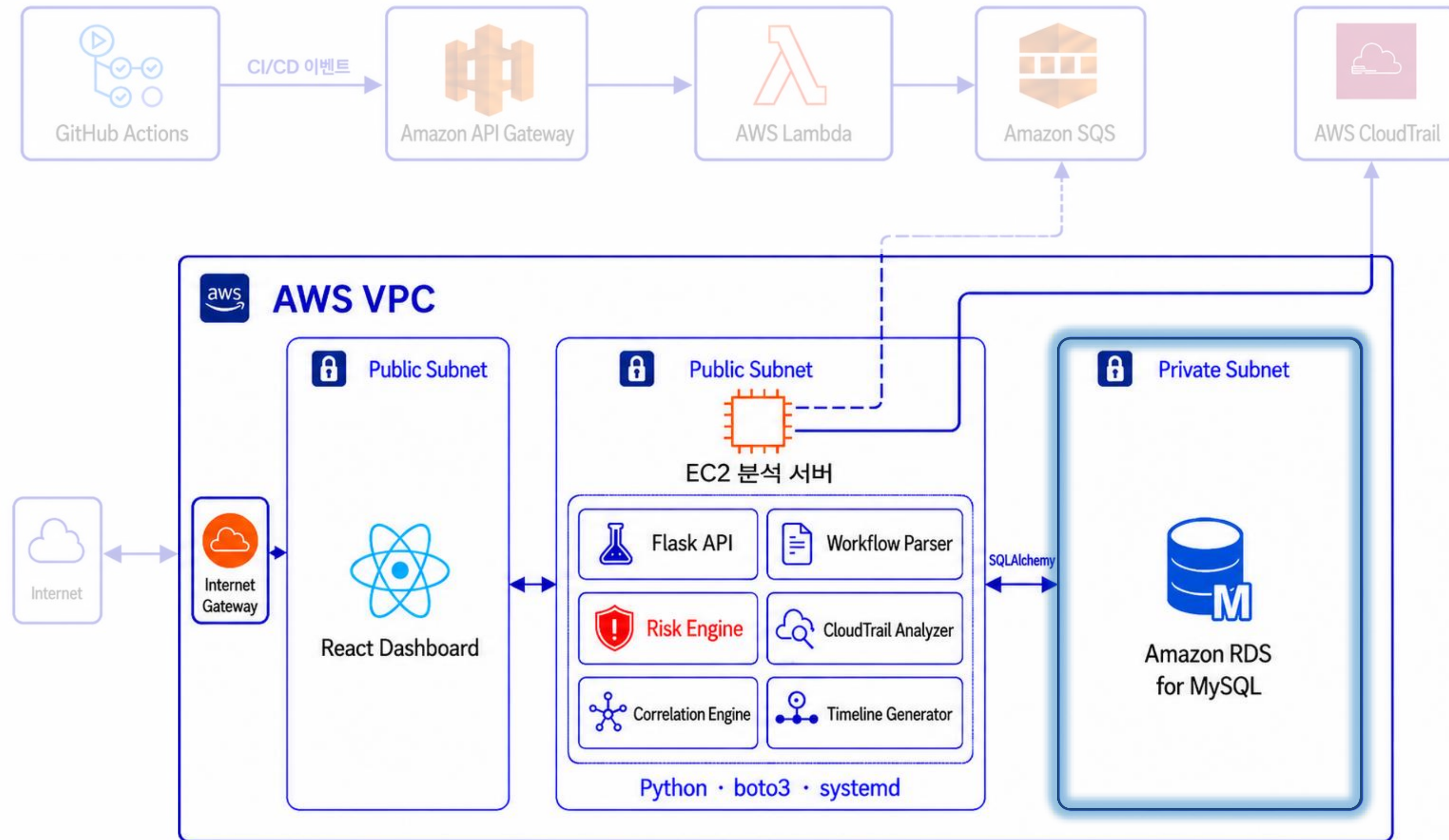
NEXTRACE 전체 아키텍처



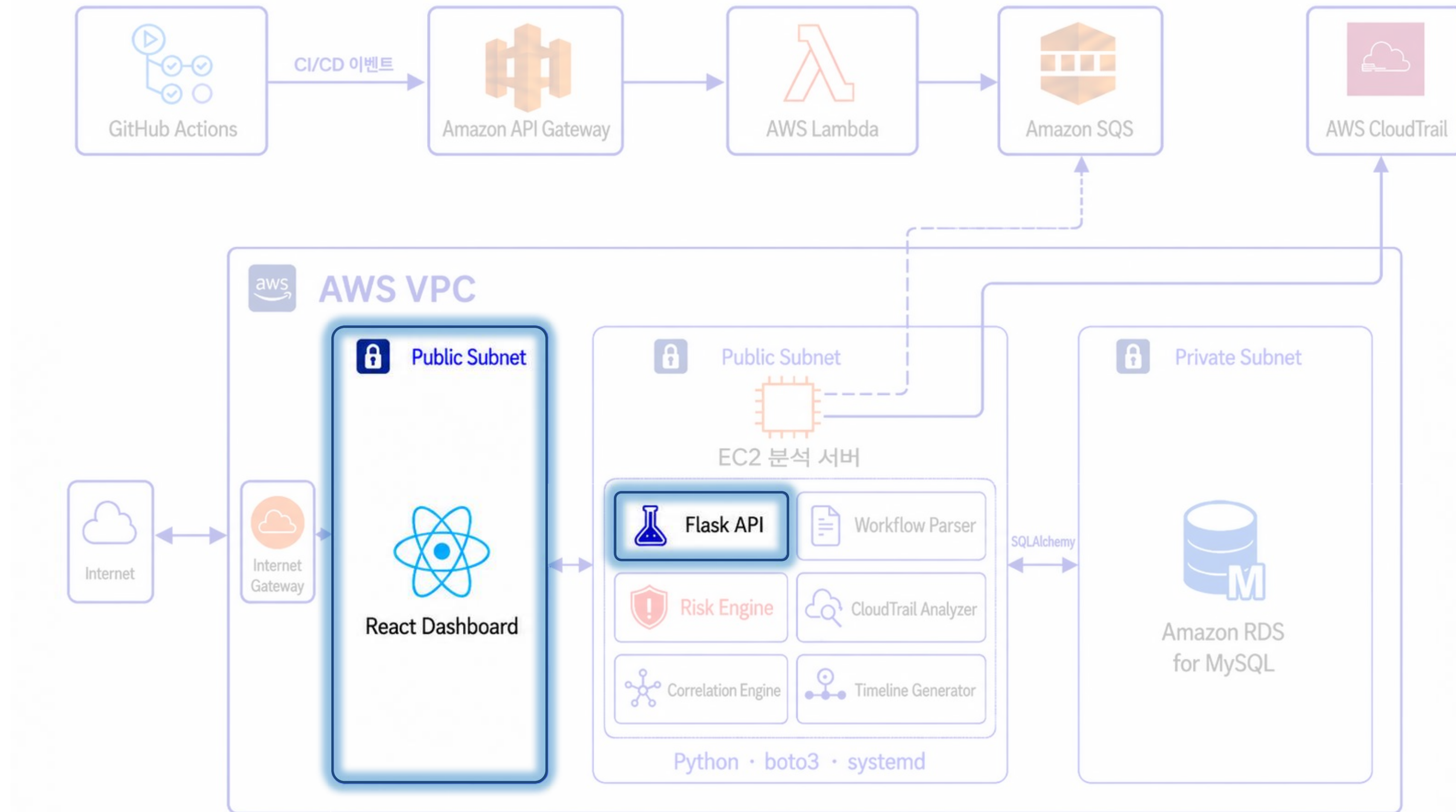
NEXTRACE 전체 아키텍처



NEXTRACE 전체 아키텍처

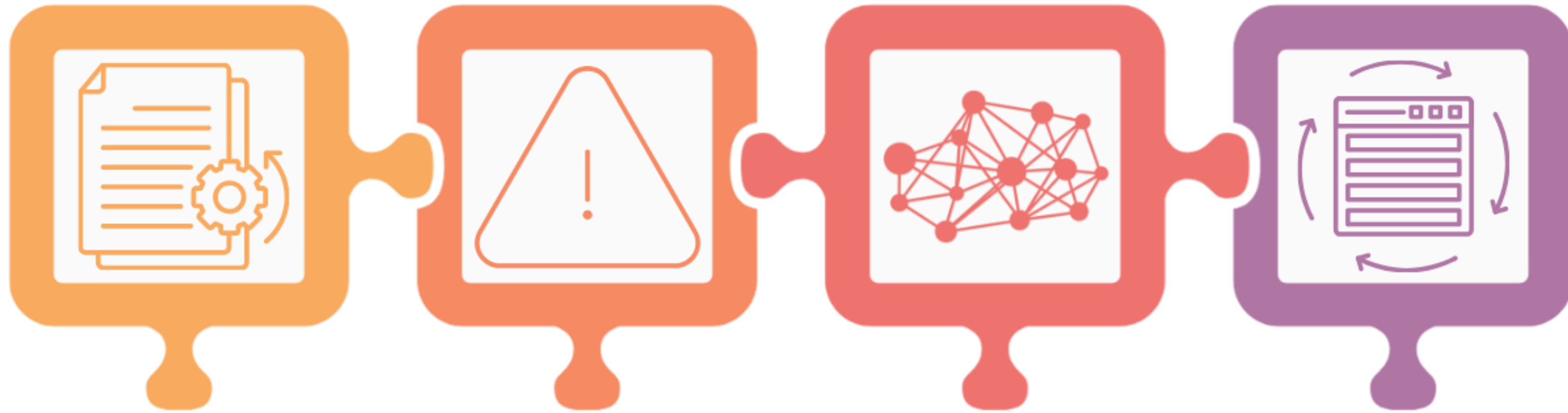


NEXTRACE 전체 아키텍처



핵심 기능

핵심 기능 침해 연관 분석 파이프라인 (4단계 + 최종 종합 분석)

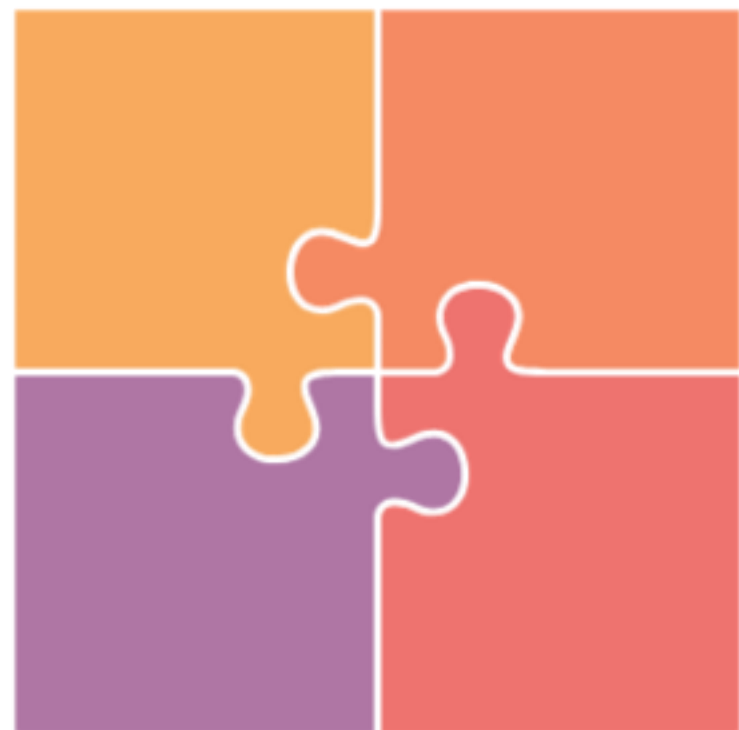


이벤트 파싱

위험 AWS Action 탐지

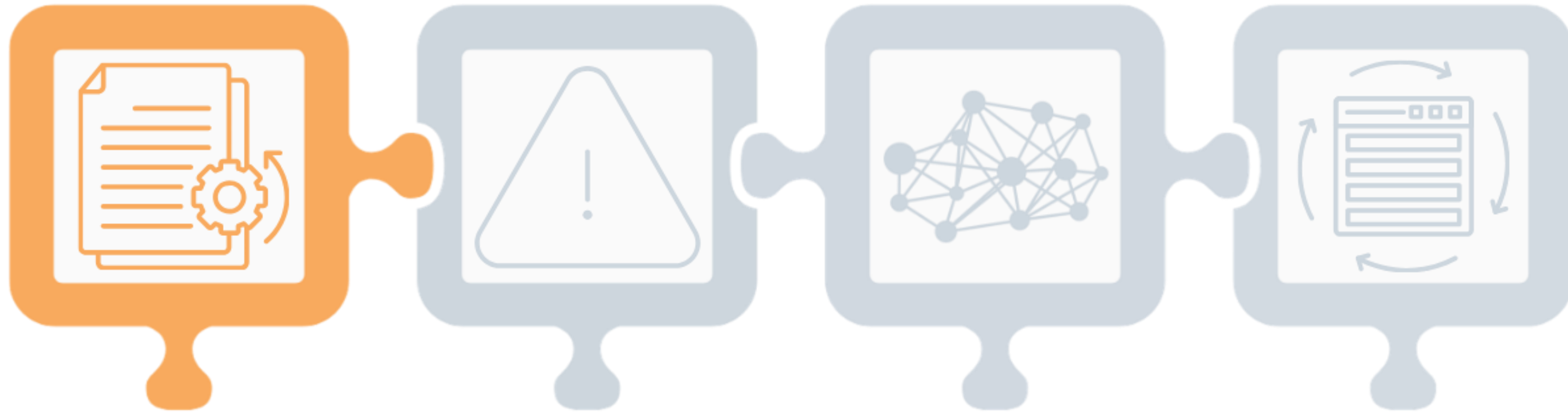
연관 · 영향 분석

공격 흐름 재구성



NEXTRACE Correlation Engine

핵심 기능 침해 연관 분석 파이프라인 (1단계)

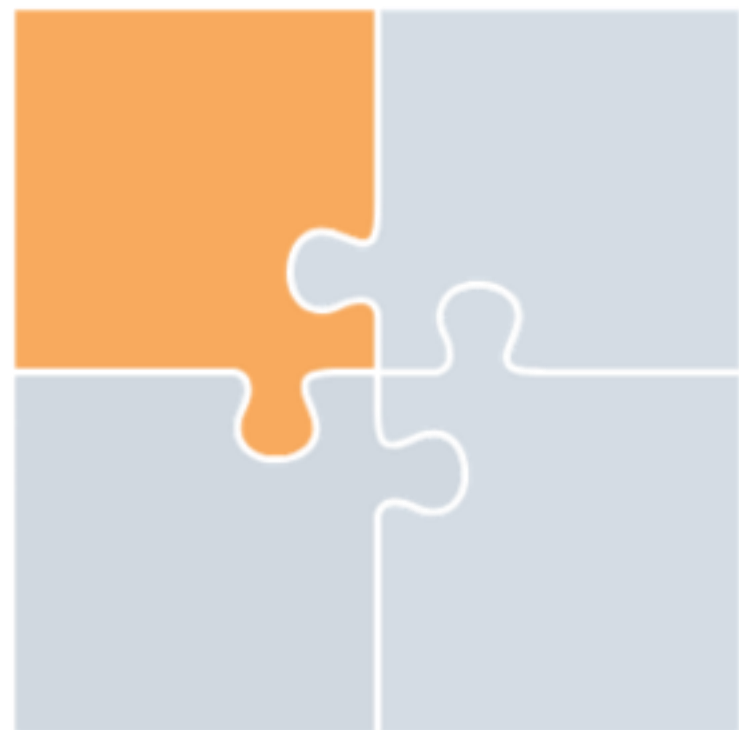


이벤트 파싱

위험 AWS Action 탐지

연관 · 영향 분석

공격 흐름 재구성



- Github Action 이벤트 수집
- Workflow · 실행 주체 정보 추출
- 분석 가능한 공통 형식으로 정규화
- SQS를 통한 분석 이벤트 전달

핵심 기능 침해 연관 분석 파이프라인 (2단계)



이벤트 파싱

위험 AWS Action 탐지

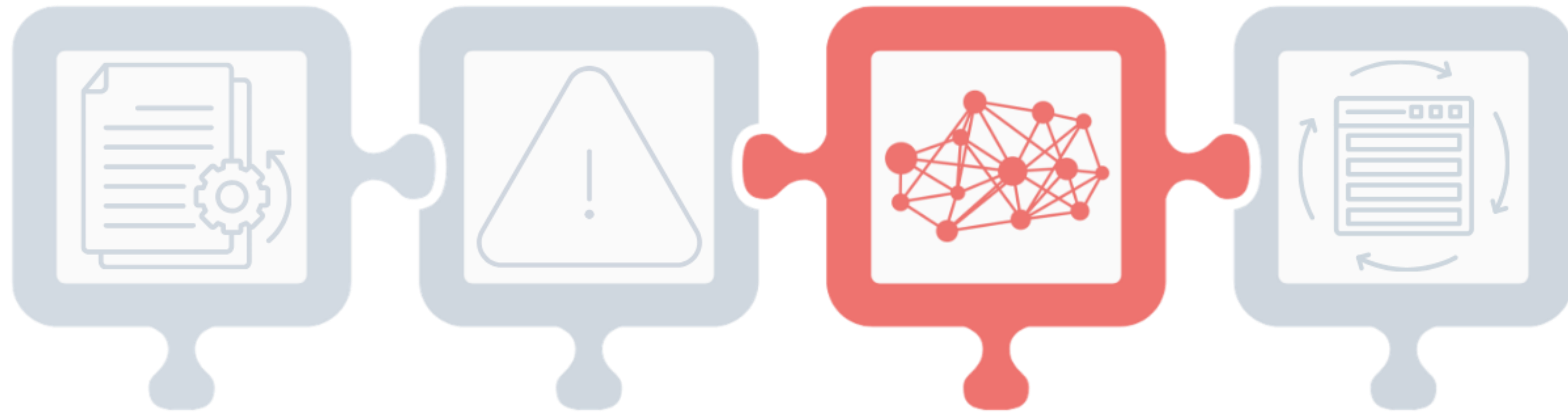
연관 · 영향 분석

공격 흐름 재구성



- CloudTrail 보안 이벤트 조회
- 위험 AWS API Action 규칙 매칭
- 자격증명 · 권한 관련 행위 식별
- 위험 이벤트와 관련 자산 추출

핵심 기능 침해 연관 분석 파이프라인 (3단계)

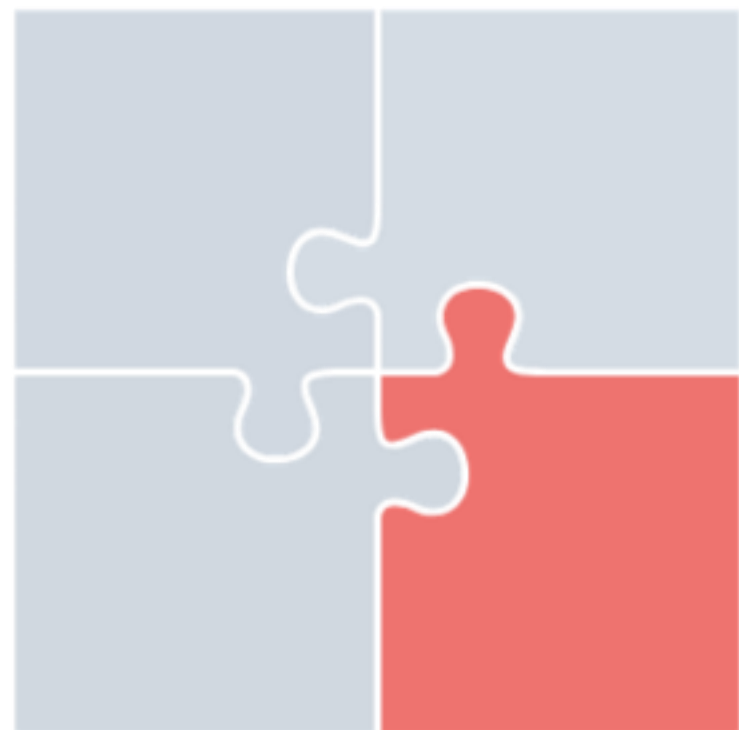


이벤트 파싱

위험 AWS Action 탐지

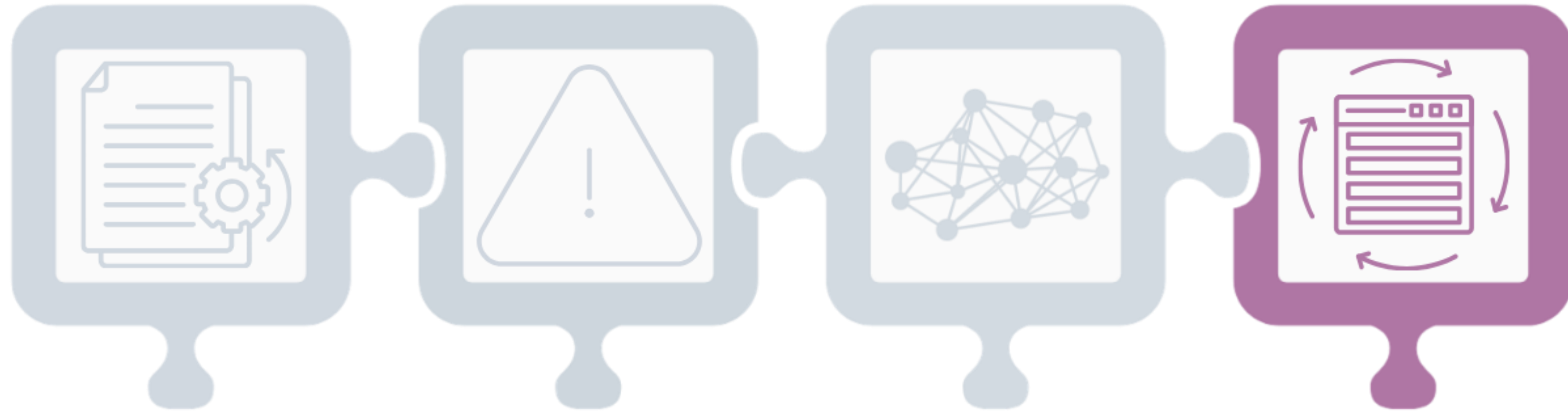
연관 · 영향 분석

공격 흐름 재구성



- CI/CD 이벤트와 CloudTrail 행위 연결
- 사용자 · 자격증명 · 시간 기반 연관 분석
- 접근하거나 변경한 AWS 자산 추적
- 실제 침해 영향 범위 분석

핵심 기능 침해 연관 분석 파이프라인 (4단계)



이벤트 파싱

위험 AWS Action 탐지

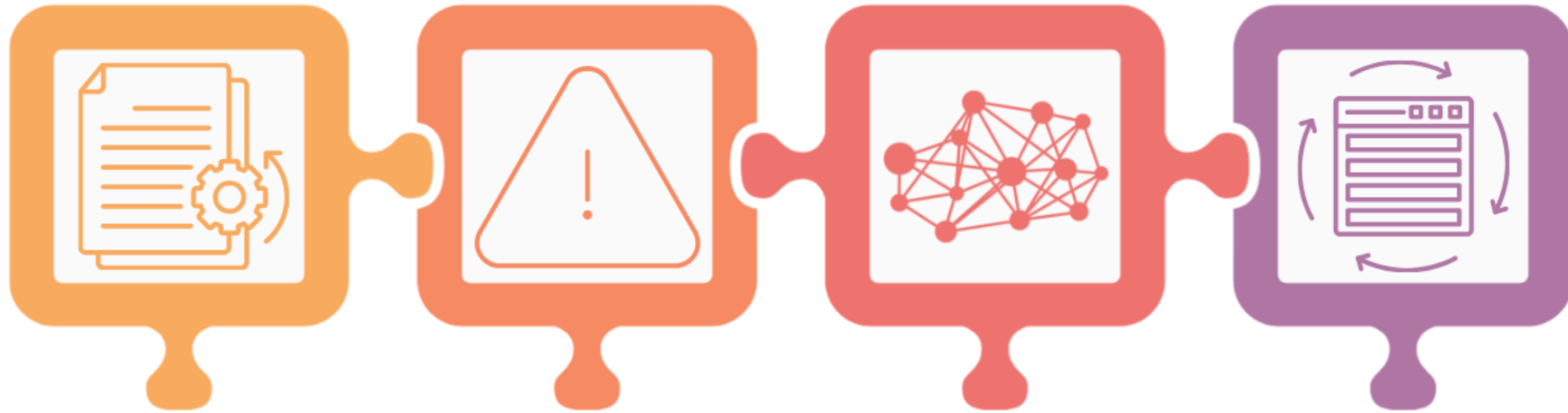
연관 · 영향 분석

공격 흐름 재구성



- 연관 이벤트를 시간순으로 정렬
- 공격 시작부터 후속 행위까지 연결
- 공격 경로와 핵심 행위 요약
- Dashboard용 타임라인 생성

핵심 기능 침해 연관 분석 파이프라인 (최종 종합 분석)

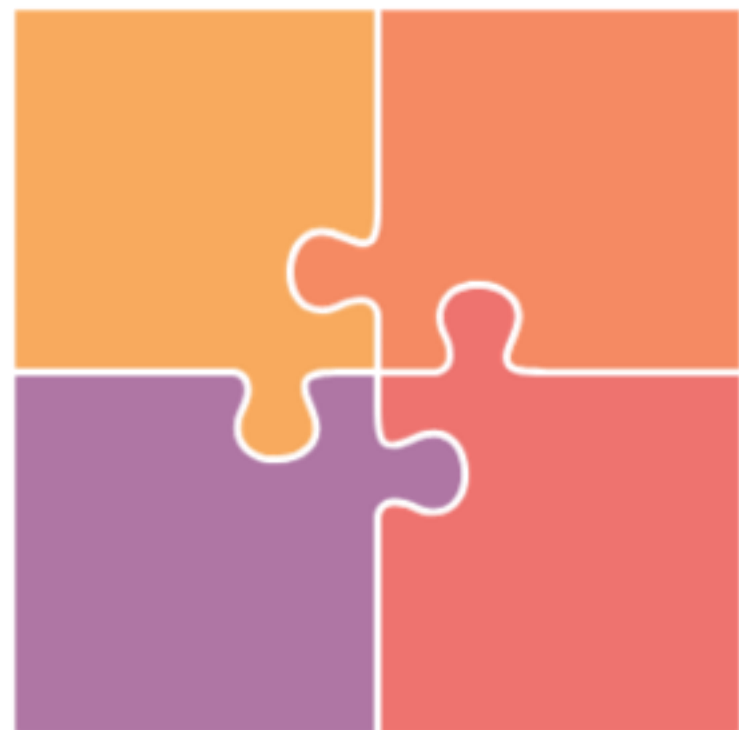


이벤트 파싱

위험 AWS Action 탐지

연관 · 영향 분석

공격 흐름 재구성



네 가지 분석 결과를 연결해 침해 흐름 종합 판단

》》》 최종 위험도 점수 · 영향 자산 · 공격 타임라인 생성

분석 로직

이벤트 수집 + 정규화

```
for region in _LOOKUP_REGIONS:          # 서울 + 버지니아(글로벌 IAM)
    client = boto3.client("cloudtrail", region_name=region)
    paginator = client.get_paginator("lookup_events")
    for page in paginator.paginate(StartTime=start, EndTime=end):
        for event in page["Events"]:
            detail = json.loads(event["CloudTrailEvent"])
            events.append({
                "event_name": event["EventName"],
                "event_source": detail.get("eventSource", ""),
                "event_time": event["EventTime"].isoformat(),
            })
```

- 두 리전의 CloudTrail 이벤트를 조회
- 이벤트명·출처·발생 시각을 목록화

```
def _to_action_name(event: dict) -> str:
    service = event.get("event_source", "").split(".")[0]
    return f"{service}:{event['event_name']}" if service else event["event_name"]
```

- CloudTrail Action 이름 표준화
- event_source + event_name 결합
- 위험 기준표 비교 형식으로 변환
(iam:CreateAccessKey)

문맥 평가 (v1.0 신규)

```
def evaluate_context(request, observed_role) -> ContextEvaluation:
    policies = load_workflow_policy()
    repository_policy = policies.get(request.repository)
    workflow_policy = repository_policy.get(request.workflow_name) if repository_policy else None

    if workflow_policy is None:
        return ContextEvaluation(policy_found=False, expected_role=None,
                                reasons=["UNREGISTERED_REPOSITORY_OR_WORKFLOW"], bonus=10)

    reasons, raw_bonus = [], 0
    if workflow_policy.get("expected_role") and observed_role != workflow_policy["expected_role"]:
        reasons.append("UNEXPECTED_ROLE"); raw_bonus += 15
    if request.event not in workflow_policy.get("allowed_events", [request.event]):
        reasons.append("UNEXPECTED_EVENT"); raw_bonus += 10
    if request.actor not in workflow_policy.get("allowed_actors", [request.actor]):
        reasons.append("UNEXPECTED_ACTOR"); raw_bonus += 5

    return ContextEvaluation(policy_found=True,
                            expected_role=workflow_policy.get("expected_role"),
                            allowed_conditional_actions=set(workflow_policy.get("allowed_conditional_actions", [])),
                            reasons=reasons, bonus=min(20, raw_bonus))
```

- Workflow 실행 문맥 검증
- Role·트리거 이벤트·실행자 확인
- 이탈 사유 기록 (**reasons**)

Finding 생성 정책 (v1.0 신규)

```
def should_create_finding(action_name, context) -> tuple[bool, str]:  
    if action_name in PRIORITY_ACTIONS:  
        return True, "PRIORITY_ACTION"  
    if action_name in CONDITIONAL_ACTIONS:  
        if context.reasons:  
            return True, "CONTEXT_MISMATCH"  
        if action_name not in context.allowed_conditional_actions:  
            return True, "ACTION_NOT_ALLOWED_BY_WORKFLOW_POLICY"  
        return False, "WORKFLOW_POLICY_MATCHED"  
    return False, "NOT_A_RISK_ACTION"
```

- 위험 Action 이중 분류
- 우선 대응 Action
: 문맥과 무관하게 Finding
- 조건부 Action
: 문맥 이탈 시에만 Finding
- 정상 배포 문맥 허용으로 오탐 방지

관측 · 매칭 분류 (observed vs matched)

```
def _classify_actions(raw_events, rules, context) -> tuple[list[ObservedAction], list[MatchedAction]]:
    observed, matched = [], []
    for e in raw_events:
        action_name = _to_action_name(e)
        create_finding, policy_result = security_policy.should_create_finding(action_name, context)

        # (1) 관측된 모든 위험 후보를 observed에 기록 (Finding 여부와 무관)
        observed.append(ObservedAction(
            action_name=action_name, event_time=e["event_time"], source_ip=e.get("source_ip"),
            user_identity_arn=e.get("user_identity_arn"), policy_result=policy_result,
        ))

        # (2) Finding으로 확정된 것만 matched에 담는다
        rule = rules.get(action_name)
        if create_finding and rule and rule.is_active:
            matched.append(MatchedAction(
                action_name=rule.action_name, category=rule.category, severity=rule.severity,
                weight=rule.weight, finding_reason=policy_result, event_time=e["event_time"],
                source_ip=e.get("source_ip"), user_identity_arn=e.get("user_identity_arn"),
            ))
    return observed, matched
```

- 위험 행동 2단계 분류
- 관측된 전체 후보 기록 (**observed_actions**)
- 확정된 Finding만 점수 반영 (**matched_actions**)
- 가중치·판정 근거 추가 (**weight, finding_reason**)

가중치 선정 이유

```
mysql> select * from risk_action_rules;
```

id	action_name	category	severity	weight	description
1	iam:CreateAccessKey	권한 상승	CRITICAL	90	장기 자격증명 신규 발급 - 영구 백도어 생성 가능
2	iam:AttachUserPolicy	권한 상승	CRITICAL	90	사용자에게 관리형 정책 직접 부착
3	iam:AttachRolePolicy	권한 상승	CRITICAL	90	역할에 관리형 정책 직접 부착
4	iam:PutUserPolicy	권한 상승	HIGH	80	사용자에 인라인 정책 삽입
5	iam:PutRolePolicy	권한 상승	HIGH	80	역할에 인라인 정책 삽입
6	iam:CreateUser	영속성	HIGH	80	신규 IAM 사용자 생성 - 백도어 계정 의심
7	iam:CreateLoginProfile	권한 상승	HIGH	75	콘솔 로그인 비밀번호 설정
8	iam:UpdateAssumeRolePolicy	권한 상승	HIGH	75	누가 이 역할을 assume 할 수 있는지 변경
9	s3:PutBucketPolicy	데이터 유출	HIGH	65	S3 버킷 정책 변경 - 외부 공개 위험
10	s3:PutBucketAcl	데이터 유출	HIGH	70	S3 버킷 ACL 변경
11	kms:Decrypt	데이터 유출	HIGH	70	암호화된 데이터 복호화
12	sts:AssumeRole	권한 상승	MEDIUM	35	다른 역할로 전환
13	secretsmanager:GetSecretValue	정찰	MEDIUM	40	Secret 값 조회
14	ec2:RunInstances	영속성	MEDIUM	45	신규 EC2 인스턴스 생성 - 채굴 / C2 서버 의심
15	s3:GetObject	데이터 유출	MEDIUM	30	S3 객체 읽기

1. CRITICAL 등급 (90점) : "클라우드 왕관을 빼앗기는 치명적 타격"

- 장기 자격증명 생성
- 관리자 권한 정책 부여

2. HIGH 등급 (65 ~ 80점) : "대규모 데이터 유출 및 은밀한 권한 조작"

- 인프라 조작
- 데이터 유출 및 정책 변경

3. MEDIUM 등급 (30 ~ 45점) : "공격의 발판 마련 및 정찰 활동"

- 비인가 파이프라인 탐지
- 복합 위협 등급 격상 · 조기 경고

위험도 점수 계산

```
def compute_risk_score(matched_actions, github_context_bonus=0) -> tuple[int, str, dict]:
    if not matched_actions:
        return 0, "LOW", {..., "final_score": 0, "policy_version": "NEXTRACE-RISK-v1.0"}

    highest_action = max(matched_actions, key=lambda a: a.weight) # 가장 높은 weight
    categories = sorted({a.category for a in matched_actions}) # 매칭된 카테고리 종류
    attack_category_bonus = min(15, max(0, len(categories) - 1) * 5) # 카테고리 다양할수록 가산
    github_context_bonus = min(20, max(0, github_context_bonus)) # 문맥 이탈 가산점

    score = min(100, highest_action.weight + attack_category_bonus + github_context_bonus)

    breakdown = {
        "highest_action": highest_action.action_name,
        "base_score": highest_action.weight,
        "categories": categories,
        "attack_category_bonus": attack_category_bonus,
        "github_context_bonus": github_context_bonus,
        "final_score": score,
        "policy_version": "NEXTRACE-RISK-v1.0",
    }
    return score, _to_level(score), breakdown
```

- v1.0 위험 점수 공식 적용
- DB 규칙 가중치 직접 반영 (**weight**)
- 공격 카테고리 조합 보너스 (최대 15점)
- Workflow 문맥 이탈 보너스 (최대 20점)
- 점수·등급·산정 근거 반환 (**score, level, breakdown**)
- 위험 등급 분류 (CRITICAL / HIGH / MEDIUM / LOW)

영향 범위 계산

```
def compute_blast_radius(raw_events: list[dict], mock: bool = True) -> BlastRadius | None:
    role_name = _first_role_name(raw_events)
    if role_name is None:
        return None
    statements = iam_policy_client.get_role_policy(role_name, mock=mock)

    exposed: list[ExposedResource] = [], defaultdict(set)
    for stmt in statements:
        if stmt.get("effect", "Allow") != "Allow":
            continue
        for resource_arn in stmt.get("resources", []):
            service = _service_of(resource_arn)
            by_service[service].add(resource_arn)
            exposed.append(ExposedResource(
                service=service, resource_arn=resource_arn, allowed_actions=list(stmt.get("actions",
                []))))

    return BlastRadius(
        role_arn=role_arn, role_name=role_name, exposed_resource_count=len(exposed),
        exposed_resources=exposed, scope_note=_scope_note(by_service),
    )
```

- IAM 정책 기반 잠재 영향 범위 계산
- Workflow의 최대 접근 권한 분석
- 접근 가능한 AWS 자산 목록화 (S3, ECR 등)
- 실제 접근 여부와 무관하게 권한 기준 평가

사건 판정

```
def determine_incident_verdict(risk_score: int, matched_action_count: int) -> str:
    if is_cloudtrail_no_finding(risk_score, matched_action_count):
        return "PASS"
    return "RISK"
```

- 정상 · 저위험 상태 명확히 구분
- 0점 + 매칭 0건만 PASS
- 매칭 발생 시 무조건 RISK
- 심각도와 사건 판정 분리 ($\text{risk_level} \neq \text{incident_verdict}$)

결과를 하나로 통합

```
observed_role = blast_radius_mod.find_observed_role_name(raw_events)
context = security_policy.evaluate_context(request, observed_role)
observed_actions, matched_actions = _classify_actions(raw_events, rules, context)
timeline = _build_timeline(request, raw_events, observed_actions)
risk_score, risk_level, score_breakdown = compute_risk_score(
    matched_actions, github_context_bonus=context.bonus,
)
blast_radius = blast_radius_mod.compute_blast_radius(raw_events, mock=not use_real_cloudtrail)
verdict = incident_verdict_mod.determine_incident_verdict(risk_score, len(matched_actions))

return AnalysisResult(
    incident_id=str(uuid.uuid4()), request_id=request.request_id,
    repository=request.repository, workflow_run_id=request.workflow_run_id,
    risk_score=risk_score, risk_level=risk_level,
    matched_actions=matched_actions, observed_actions=observed_actions,
    score_breakdown=score_breakdown, timeline=timeline,
    summary=_build_summary(request, matched_actions, risk_level),
    analyzed_at=datetime.now(timezone.utc).isoformat(), blast_radius=blast_radius,
    incident_verdict=verdict,
)
```

- 분석 파이프라인 통합 실행
- 단계별 분석 수행
(문맥 평가 → 분류 → 점수 계산
→ 영향범위 → 사건 판정)
- 문맥 이탈 가산점 반영
(**context.bonus**)
- 통합 결과 반환
(**AnalysisResult**)

저장 확정 (DB)

```
try:
    session.commit()
    log.info(f"DB 저장 완료: incident_id={result.incident_id} risk={result.risk_level}")
except IntegrityError:
    session.rollback()
    log.warning(f"request_id={result.request_id} 는 이미 처리된 요청 - 중복 저장 스킵 (정상 동작)")
```

- 분석 결과 DB 저장
- 중복 요청 자동 스킵 (**request_id UNIQUE**)

시연 영상

시연 영상

NEXTRACE 대시보드

개선 방향 및 마무리

기대 효과

보안 측면



- 오염된 공급망 공격 조기 감지
- 탈취 자격증명 추적
- 영향 자산 신속 식별

운영 측면



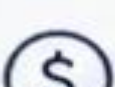
- CI/CD · CloudTrail 통합 분석
- 공격 타임라인 자동 재구성
- 침해 조사 시간 단축

사업 측면



- 클라우드 피해 최소화
- 보안 대응 비용 절감
- 감사 · 보고 근거 확보

예상 비용 (월 기준)

구분	 사양 1 (예시 규모)	 사양 2 (예시 규모)	 사양 3 (예시 규모)
 Repository	10개	100개	500개
 Workflow 실행	100회/일	1,000회/일	10,000회/일
 분석 EC2	t3.small × 1	t3.medium × 2	t3.large × 4
 RDS	db.t3.micro	db.t3.small (Multi-AZ)	db.t3.medium (Multi-AZ)
 Lambda / API Gateway	낮음	보통	높음
 CloudTrail 로그	수백 MB/일	수 GB/일	수십 GB/일
 예상 월 비용	약 \$30 ~ 50	약 \$200 ~ 400	약 \$1,000 ~ 2,000



프로젝트 실측 비용 (프로젝트 기간)

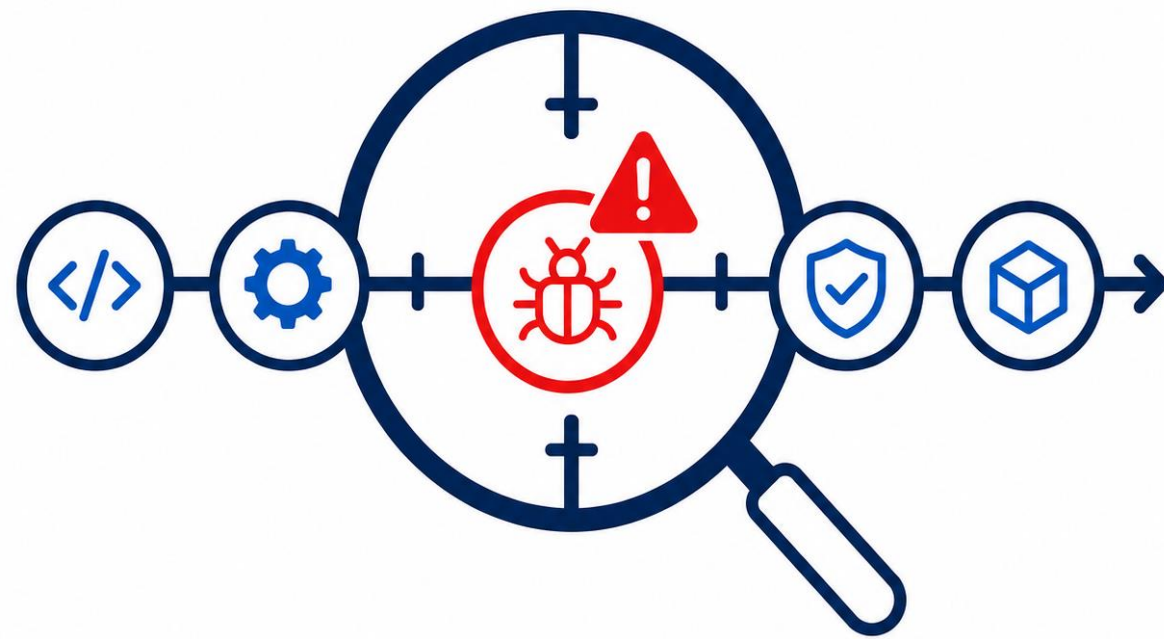


약 \$7

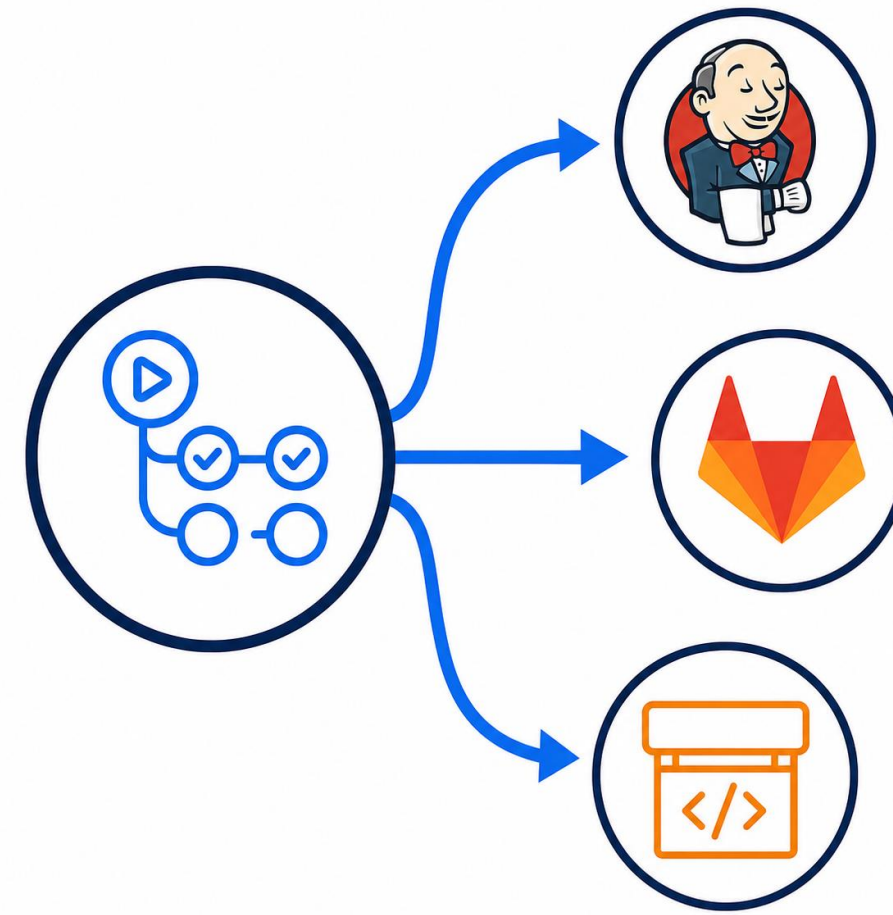
(현재 PoC 환경 기준)

* 비용은 사용량 및 트래픽, 보존 기간 등에 따라 변동될 수 있음

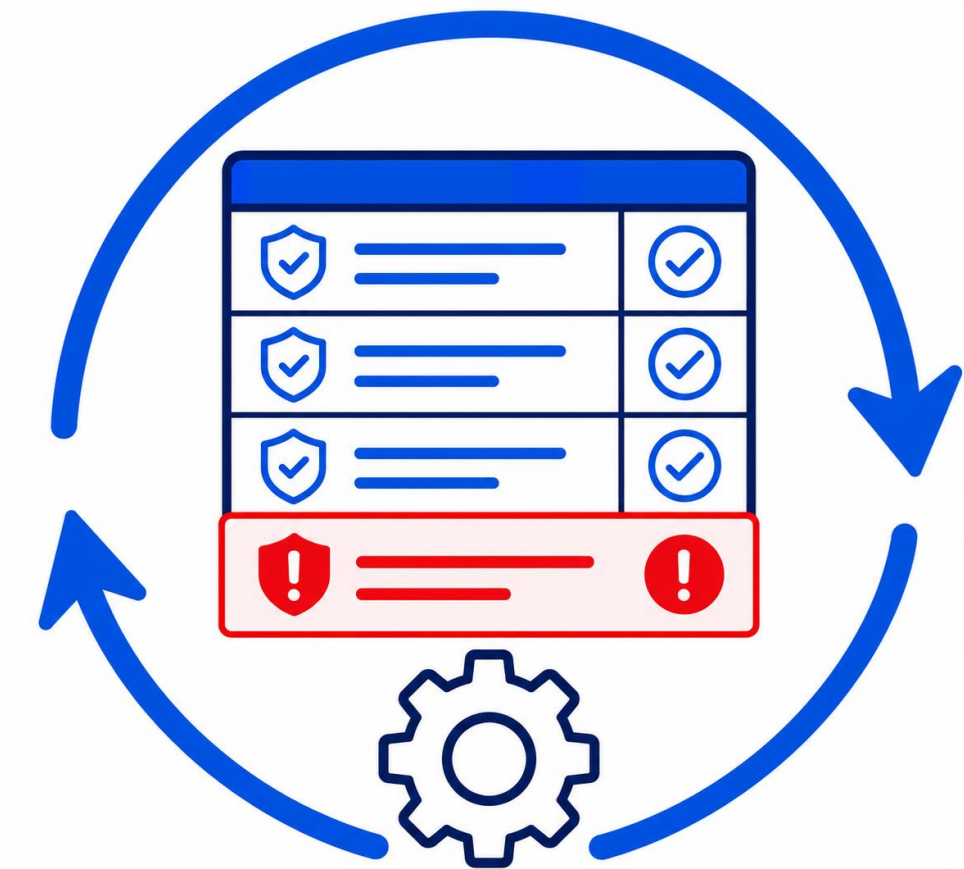
개선 방향



연동 범위 확장



탐지 정확도 향상



판단 기준 자동화

자체 평가 및 소회

자체 평가

- 최근 보안 이슈에 대응 가능한 시스템 구현
- 대시보드로 사용한 편의성 증가
- 명확한 한계점과 개선 방향

기술적 소회

- 단일 이벤트보다 상관 분석의 중요성 확인
- IAM 정책과 CloudTrail을 결합해 분석 정확도 향상
- 탐지 근거를 설명할 수 있는 규칙 기반 분석 구현

출처

- **CERT-EU / European Commission Cloud Breach: A Supply-Chain Compromise**

cert.europa.eu

활용 : 실제 CI/CD 공급망 침해 시나리오 근거

- **OWASP / Top 10 CI/CD Security Risks**

owasp.org/www-project-top-10-ci-cd-security-risks

활용 : 위험 Action 분류 및 탐지 기준

- **GITHUB DOCS / Secure Use Reference**

docs.github.com/en/actions/reference/security/secure-use

활용 : GitHub Actions 및 AWS OIDC 인증 구조

- **GITHUB DOCS / OpenID Connect**

docs.github.com/en/actions/reference/security/openid-connect

활용 : GitHub Actions 및 AWS OIDC 인증 구조

- **AWS DOCUMENTATION / Understanding CloudTrail Events**

docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-events.html

활용 : CloudTrail 분석 및 IAM 정책 평가

- **AWS DOCUMENTATION / CloudTrail Event Record Contents**

docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-event-reference-record-contents.html

활용 : CloudTrail 분석 및 IAM 정책 평가

출처

- **AWS DOCUMENTATION / IAM Policy Evaluation Logic**
docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
활용 : CloudTrail 분석 및 IAM 정책 평가
- **MITRE ATT&CK / Cloud Matrix · T1078.004 · T1098.001**
attack.mitre.org
활용 : 클라우드 공격 전술·기법 분류
- **Dailysecu**
www.dailysecu.com/news/articleView.html?idxno=164700
활용 : 프로젝트 주제 선정 근거
- **Dailysecu**
www.dailysecu.com/news/articleView.html?idxno=206086
활용 : 프로젝트 주제 선정 근거
- **KISA / 보안공지**
boho.or.kr/kr/bbs/view.do?bbsId=B0000133&nttId=72014&menuNo=205020
활용 : 국내 보안 위협 동향 참고

감사합니다

Q&A

NEXTRACE · Team 추격자